LAB EXPERIMENT REPORT - EXPERIMENT NO. 9

CUDA Fundamentals — GPU Architecture, Threads, Blocks, Grids and Basic Kernels

Course: High Performance Computing (HPC)	**Academic Year:** 2026-2027
Student Name: Kedar Somnath Dhotre	**PRN:** 23510090
Environment: Ubuntu 24.04 LTS (WSL2) / Windows 11	**GPU:** NVIDIA GeForce RTX 4060 Laptop (8GB GDDR6)

1. Aim & Objectives

To understand the fundamentals of CUDA programming and GPU-based parallel execution by studying the CUDA execution model and implementing basic CUDA kernels using threads, blocks, and grids.

Key Learning Objectives:

- Understand the basic architecture of CPU vs. GPU and the CUDA programming model.
- Identify the roles of host (CPU) and device (GPU) in CUDA applications.
- Understand threads, blocks, grids, and the CUDA thread hierarchy.
- Write and execute basic CUDA kernels with different thread/block configurations.
- Use CUDA memory management functions: cudaMalloc(), cudaMemcpy(), cudaFree().
- Use thread and block indices (threadIdx.x, blockIdx.x, blockDim.x) to distribute work.
- Implement data-parallel operations (vector addition, array squaring) using CUDA.
- Compare Sequential, OpenMP, and CUDA execution for vector workloads.
- Measure and analyze GPU performance including host-device transfer overhead.

2. Hardware & Software Environment Specifications

Component / Parameter	System Specification
Processor (CPU)	12th Gen Intel(R) Core(TM) i5-12450H (8 Physical Cores: 4P + 4E / 12 Logical Processors)

Clock Frequency	2.50 GHz Base Clock / 4.40 GHz Max Turbo Frequency
System Memory (RAM)	16.0 GB DDR4 / 3200 MHz Dual-Channel
GPU	NVIDIA GeForce RTX 3050 Laptop GPU (4095 MB GDDR6, Compute Capability 8.6, 16 SMs)
Host Operating System	Microsoft Windows 11 Home 64-bit
HPC Runtime Subsystem	WSL2 Ubuntu 24.04 LTS (Linux Kernel 6.6.87 x86_64)
CUDA Toolkit	NVIDIA CUDA Toolkit (nvcc compiler)
Compilers & Standards	GCC 15.2.0 / nvcc (-O3 -lm -Xcompiler -fopenmp)

3. Theory: CUDA Programming Model

3.1 Host and Device

In CUDA programming, the Host refers to the CPU and its main memory (RAM), while the Device refers to the GPU and its dedicated high-bandwidth memory (VRAM). The host program controls the overall application flow, allocates memory on both host and device, transfers data between them, and launches GPU kernels.

3.2 CUDA Kernel

A CUDA kernel is a function that runs on the GPU and is executed by many threads in parallel. A kernel is declared with __global__ qualifier and launched from host code using the <<<blocks, threads_per_block>>> syntax. Each thread executes the same kernel code but operates on different data elements identified by its unique thread index.

```
__global__ void kernel_name(args) {
    // GPU code
}
kernel_name<<<num_blocks, threads_per_block>>>(args);
```

3.3 Thread Hierarchy

CUDA organizes threads into a two-level hierarchy:

• Thread: The basic unit of execution. Each thread has a unique threadIdx within its block.

• Block: A group of threads that can cooperate via shared memory and synchronization barriers. Identified by blockIdx.

• Grid: A collection of blocks that execute the same kernel. The grid encompasses all threads launched

by a single kernel call.

The global thread ID for a 1D grid is computed as:
globalId = blockIdx.x × blockDim.x + threadIdx.x

This calculation is fundamental for mapping threads to data elements in data-parallel computations.

3.4 CUDA Memory Management

• cudaMalloc(void **devPtr, size_t size): Allocates memory on the GPU device.
• cudaMemcpy(dst, src, size, kind): Transfers data between host and device. Direction specified by cudaMemcpyHostToDevice or cudaMemcpyDeviceToHost.
• cudaFree(void *devPtr): Deallocates GPU device memory.

Data movement pipeline:
Host Memory → cudaMemcpy(H2D) → Device Memory → Kernel Execution → Device Result → cudaMemcpy(D2H) → Host Result

3.5 Boundary Checking

When the total number of data elements N is not an exact multiple of the block size, more threads are launched than needed. The boundary check 'if (id < N)' prevents out-of-bounds memory access. Number of blocks = ceil(N / threads_per_block).

4. Task Implementations

Task 1: Explore the GPU Environment

A CUDA program queries the GPU device properties using cudaGetDeviceProperties() to report GPU name, memory, compute capability, SM count, thread limits, and more.

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 1: Explore the GPU Environment
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Queries and displays comprehensive GPU device properties using the
 *   CUDA Runtime API (cudaGetDeviceProperties). Reports GPU name, memory,
 *   compute capability, SM count, max threads/block, max grid dimensions,
 *   warp size, clock rates, and memory bus width.
 * ============================================================================
 */

#include <stdio.h>
```

```
#include <cuda_runtime.h>

int main(int argc, char *argv[]) {
    int benchmark_mode = 0;
    if (argc > 1) benchmark_mode = atoi(argv[1]);

    int deviceCount = 0;
    cudaGetDeviceCount(&deviceCount);

    if (deviceCount == 0) {
        fprintf(stderr, "Error: No CUDA-capable GPU detected.\n");
        return 1;
    }

    for (int dev = 0; dev < deviceCount; dev++) {
        cudaDeviceProp prop;
        cudaGetDeviceProperties(&prop, dev);

        if (benchmark_mode) {
            printf("DATA_GPU_INFO|%s|%d.%d|%d|%d|%d|%lu|%d|%d|%d\n",
                   prop.name, prop.major, prop.minor,
                   prop.multiProcessorCount,
                   prop.maxThreadsPerBlock,
                   prop.maxThreadsPerMultiProcessor,
                   prop.totalGlobalMem / (1024 * 1024),
                   prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);
        } else {
            printf("======================================================================\n");
            printf("  Task 1: GPU Environment Information — Device %d\n", dev);
            printf("======================================================================\n");
            printf("  GPU Name                  : %s\n", prop.name);
            printf("  Compute Capability        : %d.%d\n", prop.major, prop.minor);
            printf("  Total Global Memory       : %lu MB (%.2f GB)\n",
                   prop.totalGlobalMem / (1024 * 1024),
                   prop.totalGlobalMem / (1024.0 * 1024.0 * 1024.0));
            printf("  Number of SMs             : %d\n", prop.multiProcessorCount);
            printf("  Max Threads per Block     : %d\n", prop.maxThreadsPerBlock);
            printf("  Max Threads per SM        : %d\n", prop.maxThreadsPerMultiProcessor);
            printf("  Warp Size                 : %d\n", prop.warpSize);
            printf("  Max Block Dimensions      : (%d, %d, %d)\n",
                   prop.maxThreadsDim[0], prop.maxThreadsDim[1], prop.maxThreadsDim[2]);
            printf("  Max Grid Dimensions       : (%d, %d, %d)\n",
                   prop.maxGridSize[0], prop.maxGridSize[1], prop.maxGridSize[2]);
            printf("  Memory Bus Width          : %d bits\n", prop.memoryBusWidth);
            printf("  L2 Cache Size             : %d KB\n", prop.l2CacheSize / 1024);
            printf("  Shared Memory per Block   : %lu KB\n", prop.sharedMemPerBlock / 1024);
            printf("  Total Constant Memory     : %lu KB\n", prop.totalConstMem / 1024);
            printf("  Device Can Map Host Memory: %s\n", prop.canMapHostMemory ? "Yes" : "No");
            printf("  Concurrent Kernels        : %s\n", prop.concurrentKernels ? "Yes" : "No");
            printf("  ECC Enabled               : %s\n", prop.ECCEnabled ? "Yes" : "No");
            printf("  Unified Addressing        : %s\n", prop.unifiedAddressing ? "Yes" : "No");
            printf("======================================================================\n");
        }
    }

    return 0;
}
```

Sample Output:

```
======================================================================
  Task 1: GPU Environment Information â€” Device 0
======================================================================
  GPU Name                  : NVIDIA GeForce RTX 3050 Laptop GPU
  Compute Capability        : 8.6
  Total Global Memory       : 4095 MB (4.00 GB)
  Number of SMs             : 16
  Max Threads per Block     : 1024
  Max Threads per SM        : 1536
  Warp Size                 : 32
  Max Block Dimensions      : (1024, 1024, 64)
  Max Grid Dimensions       : (2147483647, 65535, 65535)
  Memory Bus Width          : 128 bits
  L2 Cache Size             : 1536 KB
  Shared Memory per Block   : 48 KB
  Total Constant Memory     : 64 KB
  Device Can Map Host Memory: Yes
  Concurrent Kernels        : Yes
  ECC Enabled               : No
  Unified Addressing        : Yes
======================================================================
```

GPU Properties Summary:

Parameter	Value
GPU Name	NVIDIA GeForce RTX 3050 Laptop GPU
Total GPU Memory	4095 MB
CUDA Compute Capability	8.6
Number of SMs	16
Max Threads per Block	1024
Max Grid Dimensions	(2147483647, 65535, 65535)
Max Threads per SM	1536

Task 2: First CUDA Program — Hello from GPU

A kernel is launched where every thread prints 'Hello from GPU Thread <id>'. Tested with four configurations: (1,8), (1,16), (2,8), (4,8).

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 2: First CUDA Program – Hello from GPU
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Launches a CUDA kernel where every thread prints a greeting message.
 *   Demonstrates thread indexing with different block/thread configurations:
 *     - 1 block, 8 threads
 *     - 1 block, 16 threads
 *     - 2 blocks, 8 threads per block
 *     - 4 blocks, 8 threads per block
 * ============================================================================
 */

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void helloKernel() {
    int globalId = blockIdx.x * blockDim.x + threadIdx.x;
    printf("Hello from GPU Thread %d (Block %d, Thread %d)\n",
           globalId, blockIdx.x, threadIdx.x);
}

int main(int argc, char *argv[]) {
    int benchmark_mode = 0;
    if (argc > 1) benchmark_mode = atoi(argv[1]);

    /* Configuration table: (blocks, threads_per_block) */
    int configs[][2] = {
        {1,  8},
        {1, 16},
        {2,  8},
        {4,  8}
    };
    int num_configs = sizeof(configs) / sizeof(configs[0]);

    for (int c = 0; c < num_configs; c++) {
        int blocks = configs[c][0];
        int threads = configs[c][1];
        int total = blocks * threads;

        if (!benchmark_mode) {
            printf("============================================================================\n");
            printf("  Configuration: %d block(s), %d threads/block = %d total threads\n",
                   blocks, threads, total);
            printf("============================================================================\n");
            fflush(stdout);
        }

        helloKernel<<<blocks, threads>>>();
        cudaDeviceSynchronize();

        if (benchmark_mode) {
```

```
            printf("DATA_HELLO|%d|%d|%d\n", blocks, threads, total);
        }

        if (!benchmark_mode) {
            printf("\n");
        }
    }

    return 0;
}
```

Configuration Results:

Blocks	Threads/Block	Total Threads	Output Observation
1	8	8	Threads 0–7 print. Single block, linear IDs.
1	16	16	Threads 0–15 print. All in one block.
2	8	16	Two blocks of 8. Block 0: IDs 0–7, Block 1: IDs 8–15.
4	8	32	Four blocks of 8. Global IDs span 0–31 across blocks.

Sample Output:

```
======================================================================
  Configuration: 1 block(s), 8 threads/block = 8 total threads
======================================================================
Hello from GPU Thread 0 (Block 0, Thread 0)
Hello from GPU Thread 1 (Block 0, Thread 1)
Hello from GPU Thread 2 (Block 0, Thread 2)
Hello from GPU Thread 3 (Block 0, Thread 3)
Hello from GPU Thread 4 (Block 0, Thread 4)
Hello from GPU Thread 5 (Block 0, Thread 5)
Hello from GPU Thread 6 (Block 0, Thread 6)
Hello from GPU Thread 7 (Block 0, Thread 7)

======================================================================
  Configuration: 1 block(s), 16 threads/block = 16 total threads
======================================================================
Hello from GPU Thread 0 (Block 0, Thread 0)
Hello from GPU Thread 1 (Block 0, Thread 1)
Hello from GPU Thread 2 (Block 0, Thread 2)
Hello from GPU Thread 3 (Block 0, Thread 3)
Hello from GPU Thread 4 (Block 0, Thread 4)
```

```
Hello from GPU Thread 5 (Block 0, Thread 5)
Hello from GPU Thread 6 (Block 0, Thread 6)
Hello from GPU Thread 7 (Block 0, Thread 7)
Hello from GPU Thread 8 (Block 0, Thread 8)
Hello from GPU Thread 9 (Block 0, Thread 9)
Hello from GPU Thread 10 (Block 0, Thread 10)
Hello from GPU Thread 11 (Block 0, Thread 11)
Hello from GPU Thread 12 (Block 0, Thread 12)
Hello from GPU Thread 13 (Block 0, Thread 13)
Hello from GPU Thread 14 (Block 0, Thread 14)
Hello from GPU Thread 15 (Block 0, Thread 15)

================================================================
  Configuration: 2 block(s), 8 threads/block = 16 total threads
================================================================
Hello from GPU Thread 0 (Block 0, Thread 0)
Hello from GPU Thread 1 (Block 0, Thread 1)
Hello from GPU Thread 2 (Block 0, Thread 2)
Hello from GPU Thread 3 (Block 0, Thread 3)
Hello from GPU Thread 4 (Block 0, Thread 4)
Hello from GPU Thread 5 (Block 0, Thread 5)
Hello from GPU Thread 6 (Block 0, Thread 6)
Hello from GPU T
```

Observation: Changing the number of blocks changes the global thread ID values because globalId = blockIdx.x × blockDim.x + threadIdx.x. With more blocks, higher global IDs are generated. Thread IDs within each block (threadIdx.x) always start from 0.

Task 3: Understand Thread and Block IDs

Displays Block ID (blockIdx.x), Thread ID within Block (threadIdx.x), and Global Thread ID (blockIdx.x × blockDim.x + threadIdx.x) using 2 blocks × 4 threads.

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 3: Understand Thread and Block IDs
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Demonstrates CUDA thread hierarchy by displaying:
 *     - Block ID (blockIdx.x)
 *     - Thread ID within block (threadIdx.x)
 *     - Global Thread ID (blockIdx.x * blockDim.x + threadIdx.x)
 *   Run with 2 blocks × 4 threads per block.
 * ============================================================================
 */

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void showIdsKernel() {
    int blockId  = blockIdx.x;
```

```
    int threadId = threadIdx.x;
    int globalId = blockIdx.x * blockDim.x + threadIdx.x;

    printf("Block %d | Thread %d | Global ID %d\n",
           blockId, threadId, globalId);
}

int main(int argc, char *argv[]) {
    int benchmark_mode = 0;
    if (argc > 1) benchmark_mode = atoi(argv[1]);

    int blocks  = 2;
    int threads = 4;

    if (!benchmark_mode) {
        printf("======================================================================\n");
        printf("  Task 3: Thread and Block IDs\n");
        printf("  Configuration: %d blocks × %d threads/block = %d total threads\n",
               blocks, threads, blocks * threads);
        printf("======================================================================\n");
        printf("  Expected Global IDs: 0 through %d\n\n", blocks * threads - 1);
        fflush(stdout);
    }

    showIdsKernel<<<blocks, threads>>>();
    cudaDeviceSynchronize();

    if (benchmark_mode) {
        printf("DATA_IDS|%d|%d|%d\n", blocks, threads, blocks * threads);
    }

    if (!benchmark_mode) {
        printf("\n======================================================================\n");
        printf("  Global ID = blockIdx.x * blockDim.x + threadIdx.x\n");
        printf("  = blockIdx.x * %d + threadIdx.x\n", threads);
        printf("======================================================================\n");
    }

    return 0;
}
```

Expected Output Structure:

Block ID	Thread ID	Global ID
0	0	0
0	1	1
0	2	2

0	3	3
1	0	4
1	1	5
1	2	6
1	3	7

Sample Output:

```
================================================================
  Task 3: Thread and Block IDs
  Configuration: 2 blocks Ã— 4 threads/block = 8 total threads
================================================================
  Expected Global IDs: 0 through 7

Block 0 | Thread 0 | Global ID 0
Block 0 | Thread 1 | Global ID 1
Block 0 | Thread 2 | Global ID 2
Block 0 | Thread 3 | Global ID 3
Block 1 | Thread 0 | Global ID 4
Block 1 | Thread 1 | Global ID 5
Block 1 | Thread 2 | Global ID 6
Block 1 | Thread 3 | Global ID 7

================================================================
  Global ID = blockIdx.x * blockDim.x + threadIdx.x
  = blockIdx.x * 4 + threadIdx.x
================================================================
```

Task 4: Vector Addition Using CUDA

Implements vector addition C[i] = A[i] + B[i] in both sequential CPU and CUDA GPU versions. Each CUDA thread computes one element using global thread ID with boundary checking.

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 4: Vector Addition Using CUDA
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Implements vector addition C[i] = A[i] + B[i] in two versions:
 *     Step 1: Sequential CPU version with timing.
 *     Step 2: CUDA GPU version with timing.
 *   Includes correctness verification comparing CPU and GPU results.
```

```
 *   Uses global thread ID: id = blockIdx.x * blockDim.x + threadIdx.x
 *   with boundary check if (id < N).
 * ============================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <chrono>
#include <cuda_runtime.h>

/* ---- Sequential CPU Vector Addition ---- */
void vectorAddCPU(const float *A, const float *B, float *C, int N) {
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

/* ---- CUDA Kernel: Vector Addition ---- */
__global__ void vectorAddKernel(const float *A, const float *B, float *C, int N) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id < N) {
        C[id] = A[id] + B[id];
    }
}

/* ---- Initialize vectors with deterministic values ---- */
void initVectors(float *A, float *B, int N) {
    for (int i = 0; i < N; i++) {
        A[i] = sinf(i * 0.001f) * 100.0f;
        B[i] = cosf(i * 0.001f) * 100.0f;
    }
}

/* ---- Verify correctness ---- */
int verifyResults(const float *cpu_C, const float *gpu_C, int N, float tolerance) {
    int mismatches = 0;
    float max_diff = 0.0f;
    for (int i = 0; i < N; i++) {
        float diff = fabsf(cpu_C[i] - gpu_C[i]);
        if (diff > max_diff) max_diff = diff;
        if (diff > tolerance) {
            if (mismatches < 5)
                printf("  Mismatch at index %d: CPU=%.6f, GPU=%.6f, diff=%.6e\n",
                       i, cpu_C[i], gpu_C[i], diff);
            mismatches++;
        }
    }
    printf("  Max absolute difference: %.6e\n", max_diff);
    return mismatches;
}

int main(int argc, char *argv[]) {
    int N = 1000000;
    int threads_per_block = 256;
    int benchmark_mode = 0;

    if (argc > 1) N = atoi(argv[1]);
```

```
    if (argc > 2) threads_per_block = atoi(argv[2]);
    if (argc > 3) benchmark_mode = atoi(argv[3]);

    size_t size = N * sizeof(float);

    /* Host memory allocation */
    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C_cpu = (float *)malloc(size);
    float *h_C_gpu = (float *)malloc(size);

    if (!h_A || !h_B || !h_C_cpu || !h_C_gpu) {
        fprintf(stderr, "Host memory allocation failed for N=%d\n", N);
        return 1;
    }

    initVectors(h_A, h_B, N);

    /* ---- Step 1: Sequential CPU Version ---- */
    auto t_start = std::chrono::high_resolution_clock::now();
    vectorAddCPU(h_A, h_B, h_C_cpu, N);
    auto t_end = std::chrono::high_resolution_clock::now();
    double cpu_time_ms = std::chrono::duration<double, std::milli>(t_end - t_start).count();

    /* ---- Step 2: CUDA GPU Version ---- */
    float *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_C, size);

    /* Create CUDA events for precise timing */
    cudaEvent_t ev_start, ev_h2d_done, ev_kernel_done, ev_d2h_done;
    cudaEventCreate(&ev_start);
    cudaEventCreate(&ev_h2d_done);
    cudaEventCreate(&ev_kernel_done);
    cudaEventCreate(&ev_d2h_done);

    /* Time: Host-to-Device transfer */
    cudaEventRecord(ev_start);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    cudaEventRecord(ev_h2d_done);

    /* Time: Kernel execution */
    int num_blocks = (N + threads_per_block - 1) / threads_per_block;
    vectorAddKernel<<<num_blocks, threads_per_block>>>(d_A, d_B, d_C, N);
    cudaEventRecord(ev_kernel_done);

    /* Time: Device-to-Host transfer */
    cudaMemcpy(h_C_gpu, d_C, size, cudaMemcpyDeviceToHost);
    cudaEventRecord(ev_d2h_done);
    cudaEventSynchronize(ev_d2h_done);

    float h2d_ms = 0, kernel_ms = 0, d2h_ms = 0;
    cudaEventElapsedTime(&h2d_ms, ev_start, ev_h2d_done);
    cudaEventElapsedTime(&kernel_ms, ev_h2d_done, ev_kernel_done);
    cudaEventElapsedTime(&d2h_ms, ev_kernel_done, ev_d2h_done);
    float gpu_total_ms = h2d_ms + kernel_ms + d2h_ms;
```

```
    /* Correctness verification */
    int mismatches = verifyResults(h_C_cpu, h_C_gpu, N, 1e-4f);

    if (benchmark_mode) {
        printf("DATA_VECADD|%d|%d|%.4f|%.4f|%.4f|%.4f|%.4f|%d\n",
               N, threads_per_block, cpu_time_ms, h2d_ms, kernel_ms, d2h_ms,
               gpu_total_ms, mismatches);
    } else {
        printf("========================================================================\n");
        printf("  Task 4: Vector Addition — Sequential vs CUDA\n");
        printf("========================================================================\n");
        printf("  Vector Size (N)       : %d\n", N);
        printf("  Threads per Block     : %d\n", threads_per_block);
        printf("  Number of Blocks      : %d\n", num_blocks);
        printf("------------------------------------------------------------------------\n");
        printf("  CPU Sequential Time   : %.4f ms\n", cpu_time_ms);
        printf("------------------------------------------------------------------------\n");
        printf("  GPU Host→Device Time  : %.4f ms\n", h2d_ms);
        printf("  GPU Kernel Time       : %.4f ms\n", kernel_ms);
        printf("  GPU Device→Host Time  : %.4f ms\n", d2h_ms);
        printf("  GPU Total Time        : %.4f ms\n", gpu_total_ms);
        printf("------------------------------------------------------------------------\n");
        printf("  Speedup (CPU/GPU)     : %.2fx\n", cpu_time_ms / gpu_total_ms);
        printf("  Kernel-only Speedup   : %.2fx\n", cpu_time_ms / kernel_ms);
        printf("------------------------------------------------------------------------\n");
        printf("  Correctness Check     : %s (%d mismatches)\n",
               mismatches == 0 ? "PASSED" : "FAILED", mismatches);
        printf("========================================================================\n");
    }

    /* Cleanup */
    cudaEventDestroy(ev_start);
    cudaEventDestroy(ev_h2d_done);
    cudaEventDestroy(ev_kernel_done);
    cudaEventDestroy(ev_d2h_done);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C_cpu);
    free(h_C_gpu);

    return 0;
}
```

Performance Results:

Vector Size	CPU (ms)	H→D (ms)	Kernel (ms)	D→H (ms)	GPU Total (ms)	Speedup
100,000	0.4810	0.1860	0.1786	0.1600	0.5245	0.92x

500,000	1.1963	0.6864	0.2178	0.5754	1.4796	0.81x
1,000,000	1.8747	0.9860	0.2161	1.0093	2.2113	0.85x
5,000,000	10.6590	5.0230	0.5261	5.4212	10.9703	0.97x
10,000,000	24.6800	11.2707	1.0306	10.0558	22.3571	1.10x

Sample Output:

```
Max absolute difference: 0.000000e+00
======================================================================
  Task 4: Vector Addition â€” Sequential vs CUDA
======================================================================
  Vector Size (N)       : 1000000
  Threads per Block     : 256
  Number of Blocks      : 3907
----------------------------------------------------------------------
  CPU Sequential Time   : 3.8113 ms
----------------------------------------------------------------------
  GPU Hostâ†’Device Time  : 1.0699 ms
  GPU Kernel Time       : 0.2398 ms
  GPU Deviceâ†’Host Time  : 1.0569 ms
  GPU Total Time        : 2.3666 ms
----------------------------------------------------------------------
  Speedup (CPU/GPU)     : 1.61x
  Kernel-only Speedup   : 15.89x
----------------------------------------------------------------------
  Correctness Check     : PASSED (0 mismatches)
======================================================================
```

Task 5: CUDA Memory Management

Implements vector addition using explicit CUDA device memory management. Separately times cudaMalloc(), cudaMemcpy(H2D), kernel execution, and cudaMemcpy(D2H).

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 5: CUDA Memory Management
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Implements vector addition using CUDA device memory with explicit
 *   management of: cudaMalloc(), cudaMemcpy(), cudaFree().
 *   Demonstrates the data movement pipeline:
 *     Host Memory → cudaMemcpy(H2D) → Device Memory → Kernel →
 *     Device Result → cudaMemcpy(D2H) → Host Result
 *   Separately times each phase and verifies correctness.
```

```
 * ============================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <cuda_runtime.h>

/* ---- CUDA Kernel: Vector Addition ---- */
__global__ void vectorAddKernel(const float *A, const float *B, float *C, int N) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id < N) {
        C[id] = A[id] + B[id];
    }
}

/* ---- Sequential CPU Version ---- */
void vectorAddCPU(const float *A, const float *B, float *C, int N) {
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

int main(int argc, char *argv[]) {
    int N = 1000000;
    int benchmark_mode = 0;

    if (argc > 1) N = atoi(argv[1]);
    if (argc > 2) benchmark_mode = atoi(argv[2]);

    size_t size = N * sizeof(float);
    int threads_per_block = 256;
    int num_blocks = (N + threads_per_block - 1) / threads_per_block;

    /* Step 1: Allocate Host Memory */
    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C_cpu = (float *)malloc(size);
    float *h_C_gpu = (float *)malloc(size);

    if (!h_A || !h_B || !h_C_cpu || !h_C_gpu) {
        fprintf(stderr, "Host memory allocation failed.\n");
        return 1;
    }

    /* Initialize host vectors */
    for (int i = 0; i < N; i++) {
        h_A[i] = sinf(i * 0.0005f) * 50.0f;
        h_B[i] = cosf(i * 0.0005f) * 50.0f;
    }

    /* CPU sequential computation for verification */
    vectorAddCPU(h_A, h_B, h_C_cpu, N);

    /* Step 2: Allocate Device Memory using cudaMalloc() */
    float *d_A, *d_B, *d_C;
```

```
    cudaEvent_t ev_alloc_start, ev_alloc_end;
    cudaEventCreate(&ev_alloc_start);
    cudaEventCreate(&ev_alloc_end);

    cudaEventRecord(ev_alloc_start);
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_C, size);
    cudaEventRecord(ev_alloc_end);
    cudaEventSynchronize(ev_alloc_end);

    float alloc_ms = 0;
    cudaEventElapsedTime(&alloc_ms, ev_alloc_start, ev_alloc_end);

    /* Step 3: Host-to-Device Transfer using cudaMemcpy() */
    cudaEvent_t ev_h2d_start, ev_h2d_end;
    cudaEventCreate(&ev_h2d_start);
    cudaEventCreate(&ev_h2d_end);

    cudaEventRecord(ev_h2d_start);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    cudaEventRecord(ev_h2d_end);
    cudaEventSynchronize(ev_h2d_end);

    float h2d_ms = 0;
    cudaEventElapsedTime(&h2d_ms, ev_h2d_start, ev_h2d_end);

    /* Step 4: Kernel Execution on Device */
    cudaEvent_t ev_kernel_start, ev_kernel_end;
    cudaEventCreate(&ev_kernel_start);
    cudaEventCreate(&ev_kernel_end);

    cudaEventRecord(ev_kernel_start);
    vectorAddKernel<<<num_blocks, threads_per_block>>>(d_A, d_B, d_C, N);
    cudaEventRecord(ev_kernel_end);
    cudaEventSynchronize(ev_kernel_end);

    float kernel_ms = 0;
    cudaEventElapsedTime(&kernel_ms, ev_kernel_start, ev_kernel_end);

    /* Step 5: Device-to-Host Transfer using cudaMemcpy() */
    cudaEvent_t ev_d2h_start, ev_d2h_end;
    cudaEventCreate(&ev_d2h_start);
    cudaEventCreate(&ev_d2h_end);

    cudaEventRecord(ev_d2h_start);
    cudaMemcpy(h_C_gpu, d_C, size, cudaMemcpyDeviceToHost);
    cudaEventRecord(ev_d2h_end);
    cudaEventSynchronize(ev_d2h_end);

    float d2h_ms = 0;
    cudaEventElapsedTime(&d2h_ms, ev_d2h_start, ev_d2h_end);

    /* Step 6: Correctness Verification */
    int mismatches = 0;
    float max_diff = 0.0f;
    for (int i = 0; i < N; i++) {
```

```
        float diff = fabsf(h_C_cpu[i] - h_C_gpu[i]);
        if (diff > max_diff) max_diff = diff;
        if (diff > 1e-4f) mismatches++;
    }

    float total_gpu_ms = alloc_ms + h2d_ms + kernel_ms + d2h_ms;

    if (benchmark_mode) {
        printf("DATA_MEMMAN|%d|%.4f|%.4f|%.4f|%.4f|%.4f|%d|%.6e\n",
               N, alloc_ms, h2d_ms, kernel_ms, d2h_ms, total_gpu_ms,
               mismatches, max_diff);
    } else {
        printf("========================================================================\n");
        printf("  Task 5: CUDA Memory Management — Detailed Phase Timing\n");
        printf("========================================================================\n");
        printf("  Vector Size (N)       : %d\n", N);
        printf("  Data Size             : %.2f MB\n", (3.0 * size) / (1024 * 1024));
        printf("------------------------------------------------------------------------\n");
        printf("  Phase Breakdown:\n");
        printf("    cudaMalloc()        : %.4f ms  [Device memory allocation]\n", alloc_ms);
        printf("    cudaMemcpy(H→D)     : %.4f ms  [Host → Device transfer]\n", h2d_ms);
        printf("    Kernel Execution    : %.4f ms  [GPU computation]\n", kernel_ms);
        printf("    cudaMemcpy(D→H)     : %.4f ms  [Device → Host transfer]\n", d2h_ms);
        printf("    TOTAL GPU Pipeline  : %.4f ms\n", total_gpu_ms);
        printf("------------------------------------------------------------------------\n");
        printf("  Transfer Overhead     : %.2f%% of total GPU time\n",
               (h2d_ms + d2h_ms) / total_gpu_ms * 100.0);
        printf("  Computation Fraction  : %.2f%% of total GPU time\n",
               kernel_ms / total_gpu_ms * 100.0);
        printf("------------------------------------------------------------------------\n");
        printf("  Correctness Check     : %s\n",
               mismatches == 0 ? "PASSED (all elements match)" : "FAILED");
        printf("  Max Absolute Diff     : %.6e\n", max_diff);
        printf("  Mismatches            : %d / %d\n", mismatches, N);
        printf("========================================================================\n");
    }

    /* Step 7: Free Device Memory using cudaFree() */
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    cudaEventDestroy(ev_alloc_start);
    cudaEventDestroy(ev_alloc_end);
    cudaEventDestroy(ev_h2d_start);
    cudaEventDestroy(ev_h2d_end);
    cudaEventDestroy(ev_kernel_start);
    cudaEventDestroy(ev_kernel_end);
    cudaEventDestroy(ev_d2h_start);
    cudaEventDestroy(ev_d2h_end);

    /* Free Host Memory */
    free(h_A);
    free(h_B);
    free(h_C_cpu);
    free(h_C_gpu);

    return 0;
```

```
}
```

Phase Timing Results:

Vector Size	cudaMalloc (ms)	H→D (ms)	Kernel (ms)	D→H (ms)	Total (ms)
100,000	0.4374	0.1991	0.2114	0.1538	1.0017
500,000	0.6380	0.5452	0.2176	0.5152	1.9160
1,000,000	0.7362	1.1168	0.2825	1.0365	3.1720
5,000,000	2.4361	4.8372	0.5907	4.5169	12.3809
10,000,000	4.4902	14.1104	1.2463	15.9464	35.7933

Memory Management Functions:

Function	Location	Purpose
cudaMalloc()	Device	Allocates memory on GPU global memory
cudaMemcpy(H→D)	Host → Device	Copies input data from CPU RAM to GPU memory
cudaMemcpy(D→H)	Device → Host	Copies computed results from GPU memory to CPU RAM

Sample Output:

```
======================================================================
  Task 5: CUDA Memory Management â€" Detailed Phase Timing
======================================================================
  Vector Size (N)       : 1000000
  Data Size             : 11.44 MB
----------------------------------------------------------------------
  Phase Breakdown:
    cudaMalloc()        : 0.9134 ms  [Device memory allocation]
    cudaMemcpy(Hâ†'D)     : 1.0295 ms  [Host â†' Device transfer]
    Kernel Execution    : 0.3018 ms  [GPU computation]
    cudaMemcpy(Dâ†'H)     : 1.0152 ms  [Device â†' Host transfer]
    TOTAL GPU Pipeline  : 3.2598 ms
----------------------------------------------------------------------
```

```
  Transfer Overhead     : 62.72% of total GPU time
  Computation Fraction  : 9.26% of total GPU time
----------------------------------------------------------------------
  Correctness Check     : PASSED (all elements match)
  Max Absolute Diff     : 0.000000e+00
  Mismatches            : 0 / 1000000
======================================================================
```

Task 6: Vector Addition with Different Block Configurations

Runs vector addition with fixed vector size using block sizes: 64, 128, 256, 512 threads/block. Analyzes the impact of block size on kernel execution time.

Source Code:

```
/*
 * ================================================================================
 * HPC Experiment 9 - Task 6: Vector Addition with Different Block Configurations
 * ================================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Runs vector addition with a fixed vector size (N = 1,000,000) using
 *   different block sizes: 64, 128, 256, 512 threads/block.
 *   Calculates the number of blocks required for each configuration.
 *   Analyzes the impact of block size on kernel execution time.
 *   Includes boundary check (id < N) to handle non-exact multiples.
 * ================================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <chrono>
#include <cuda_runtime.h>

/* ---- CUDA Kernel: Vector Addition ---- */
__global__ void vectorAddKernel(const float *A, const float *B, float *C, int N) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id < N) {
        C[id] = A[id] + B[id];
    }
}

/* ---- Sequential CPU Version ---- */
void vectorAddCPU(const float *A, const float *B, float *C, int N) {
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

int main(int argc, char *argv[]) {
    int N = 1000000;
    int benchmark_mode = 0;
```

```
    if (argc > 1) N = atoi(argv[1]);
    if (argc > 2) benchmark_mode = atoi(argv[2]);

    size_t size = N * sizeof(float);

    /* Host allocations */
    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C_cpu = (float *)malloc(size);
    float *h_C_gpu = (float *)malloc(size);

    if (!h_A || !h_B || !h_C_cpu || !h_C_gpu) {
        fprintf(stderr, "Host memory allocation failed.\n");
        return 1;
    }

    /* Initialize */
    for (int i = 0; i < N; i++) {
        h_A[i] = sinf(i * 0.001f) * 100.0f;
        h_B[i] = cosf(i * 0.001f) * 100.0f;
    }

    /* CPU sequential baseline */
    auto t_start = std::chrono::high_resolution_clock::now();
    vectorAddCPU(h_A, h_B, h_C_cpu, N);
    auto t_end = std::chrono::high_resolution_clock::now();
    double cpu_time_ms = std::chrono::duration<double, std::milli>(t_end - t_start).count();

    /* Device allocations */
    float *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_C, size);

    /* Copy to device once */
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    /* Block size configurations */
    int block_sizes[] = {64, 128, 256, 512};
    int num_configs = sizeof(block_sizes) / sizeof(block_sizes[0]);

    if (!benchmark_mode) {
        printf("======================================================================\n");
        printf("  Task 6: Vector Addition — Block Size Comparison\n");
        printf("  Vector Size (N) = %d | CPU Time = %.4f ms\n", N, cpu_time_ms);
        printf("======================================================================\n");
        printf("  %-14s %-10s %-14s %-14s %-10s\n",
               "Threads/Block", "Blocks", "Kernel (ms)", "Total (ms)", "Speedup");
        printf("  %-14s %-10s %-14s %-14s %-10s\n",
               "--------------", "------", "------------", "----------", "-------");
    }

    for (int c = 0; c < num_configs; c++) {
        int tpb = block_sizes[c];
        int num_blocks = (N + tpb - 1) / tpb;

        /* Warm-up run */
```

```
        vectorAddKernel<<<num_blocks, tpb>>>(d_A, d_B, d_C, N);
        cudaDeviceSynchronize();

        /* Timed run (average of 5 iterations) */
        cudaEvent_t ev_start, ev_end;
        cudaEventCreate(&ev_start);
        cudaEventCreate(&ev_end);

        int runs = 5;
        float total_kernel_ms = 0.0f;

        for (int r = 0; r < runs; r++) {
            cudaEventRecord(ev_start);
            vectorAddKernel<<<num_blocks, tpb>>>(d_A, d_B, d_C, N);
            cudaEventRecord(ev_end);
            cudaEventSynchronize(ev_end);
            float elapsed = 0;
            cudaEventElapsedTime(&elapsed, ev_start, ev_end);
            total_kernel_ms += elapsed;
        }

        float avg_kernel_ms = total_kernel_ms / runs;

        /* Include H2D and D2H for total time */
        cudaEvent_t ev_total_start, ev_total_end;
        cudaEventCreate(&ev_total_start);
        cudaEventCreate(&ev_total_end);

        cudaEventRecord(ev_total_start);
        cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
        cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
        vectorAddKernel<<<num_blocks, tpb>>>(d_A, d_B, d_C, N);
        cudaMemcpy(h_C_gpu, d_C, size, cudaMemcpyDeviceToHost);
        cudaEventRecord(ev_total_end);
        cudaEventSynchronize(ev_total_end);

        float total_ms = 0;
        cudaEventElapsedTime(&total_ms, ev_total_start, ev_total_end);

        /* Verify correctness */
        int mismatches = 0;
        for (int i = 0; i < N; i++) {
            if (fabsf(h_C_cpu[i] - h_C_gpu[i]) > 1e-4f) mismatches++;
        }

        float speedup = cpu_time_ms / total_ms;

        if (benchmark_mode) {
            printf("DATA_BLOCKSIZE|%d|%d|%d|%.4f|%.4f|%.4f|%d\n",
                   N, tpb, num_blocks, avg_kernel_ms, total_ms, (float)cpu_time_ms, mismatches);
        } else {
            printf("  %-14d %-10d %-14.4f %-14.4f %-10.2f  %s\n",
                   tpb, num_blocks, avg_kernel_ms, total_ms, speedup,
                   mismatches == 0 ? "✓" : "✗");
        }

        cudaEventDestroy(ev_start);
```

```
        cudaEventDestroy(ev_end);
        cudaEventDestroy(ev_total_start);
        cudaEventDestroy(ev_total_end);
    }

    if (!benchmark_mode) {
        printf("================================================================\n");
        printf("  Note: Blocks = ceil(N / threads_per_block)\n");
        printf("  Boundary check (id < N) handles N not divisible by block size.\n");
        printf("================================================================\n");
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C_cpu);
    free(h_C_gpu);

    return 0;
}
```

Block Configuration Results:

Vector Size: N = 1,000,000

Threads/Block	Blocks	Kernel (ms)	Total (ms)	Speedup
64	15625	0.0745	2.3220	0.91x
128	7813	0.0731	1.5668	1.35x
256	3907	0.0731	1.5287	1.38x
512	1954	0.0737	1.6014	1.32x

Vector Size: N = 5,000,000

Threads/Block	Blocks	Kernel (ms)	Total (ms)	Speedup
64	78125	0.3387	9.8160	1.06x
128	39063	0.3315	7.3103	1.42x

256	19532	0.3362	7.4441	1.40x
512	9766	0.3323	7.2574	1.43x

Vector Size: N = 10,000,000

Threads/Block	Blocks	Kernel (ms)	Total (ms)	Speedup
64	156250	0.6721	19.3684	2.31x
128	78125	0.6657	14.3874	3.12x
256	39063	0.6839	15.0699	2.97x
512	19532	0.6578	15.8273	2.83x

Analysis:

• Increasing threads per block does not always improve performance. There is typically an optimal block size (often 128 or 256) that balances occupancy and resource utilization.
• When N is not divisible by block size, extra threads are launched but the boundary check (id < N) prevents them from accessing invalid memory locations.
• The boundary check is essential because ceil(N/blockSize) × blockSize ≥ N, meaning some threads in the last block have IDs beyond the array bounds.

Sample Output:

```
======================================================================
  Task 6: Vector Addition â€” Block Size Comparison
  Vector Size (N) = 1000000 | CPU Time = 2.6255 ms
======================================================================
  Threads/Block  Blocks     Kernel (ms)    Total (ms)     Speedup
  -------------  ------     -----------    ----------     -------
  64             15625      0.1122         2.4079         1.09       â€“
  128            7813       0.1011         2.2420         1.17       â€“
  256            3907       0.0915         1.7412         1.51       â€“
  512            1954       0.0966         1.9318         1.36       â€“
======================================================================
  Note: Blocks = ceil(N / threads_per_block)
  Boundary check (id < N) handles N not divisible by block size.
======================================================================
```

Task 7: Vector Addition — Sequential vs OpenMP vs CUDA

Implements vector addition using three approaches (Sequential, OpenMP, CUDA) and compares execution time across vector sizes: 10^5, 10^6, 10^7, 10^8.

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 7: Sequential vs OpenMP vs CUDA Vector Addition
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Implements vector addition using three approaches:
 *     Version 1: Sequential C/C++
 *     Version 2: OpenMP (shared-memory parallelism)
 *     Version 3: CUDA (GPU parallelism)
 *   Compares execution time across vector sizes: 10^5, 10^6, 10^7, 10^8.
 *   Verifies that all three produce identical results.
 *
 * Compile: nvcc -Xcompiler -fopenmp task7_seq_omp_cuda.cu -o task7_seq_omp_cuda -lm
 * ============================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <chrono>
#include <omp.h>
#include <cuda_runtime.h>

/* ---- Sequential Version ---- */
void vectorAddSeq(const float *A, const float *B, float *C, int N) {
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

/* ---- OpenMP Version ---- */
void vectorAddOMP(const float *A, const float *B, float *C, int N) {
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

/* ---- CUDA Kernel ---- */
__global__ void vectorAddKernel(const float *A, const float *B, float *C, int N) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id < N) {
        C[id] = A[id] + B[id];
    }
}

/* ---- Verify results match ---- */
```

```
int verifyAll(const float *C_seq, const float *C_omp, const float *C_cuda, int N) {
    int errors = 0;
    for (int i = 0; i < N; i++) {
        if (fabsf(C_seq[i] - C_omp[i]) > 1e-4f ||
            fabsf(C_seq[i] - C_cuda[i]) > 1e-4f) {
            errors++;
        }
    }
    return errors;
}

int main(int argc, char *argv[]) {
    int benchmark_mode = 0;
    if (argc > 1) benchmark_mode = atoi(argv[1]);

    long sizes[] = {100000L, 1000000L, 10000000L, 100000000L};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
    int threads_per_block = 256;

    if (!benchmark_mode) {
        printf("======================================================================\n");
        printf("  Task 7: Vector Addition — Sequential vs OpenMP vs CUDA\n");
        printf("  OpenMP Threads: %d\n", omp_get_max_threads());
        printf("======================================================================\n");
        printf("  %-12s %-12s %-12s %-12s %-10s %-10s %s\n",
               "Size", "Seq (ms)", "OMP (ms)", "CUDA (ms)",
               "OMP Spdup", "CUDA Spdup", "Verify");
        printf("  %-12s %-12s %-12s %-12s %-10s %-10s %s\n",
               "----", "--------", "--------", "---------",
               "---------", "----------", "------");
    }

    for (int s = 0; s < num_sizes; s++) {
        long N = sizes[s];
        size_t size = N * sizeof(float);

        /* Check available memory - skip sizes that are too large */
        float *h_A = (float *)malloc(size);
        float *h_B = (float *)malloc(size);
        float *h_C_seq  = (float *)malloc(size);
        float *h_C_omp  = (float *)malloc(size);
        float *h_C_cuda = (float *)malloc(size);

        if (!h_A || !h_B || !h_C_seq || !h_C_omp || !h_C_cuda) {
            if (!benchmark_mode)
                printf("  %-12ld Skipped (insufficient host memory)\n", N);
            free(h_A); free(h_B); free(h_C_seq); free(h_C_omp); free(h_C_cuda);
            continue;
        }

        /* Initialize */
        for (long i = 0; i < N; i++) {
            h_A[i] = sinf(i * 0.0001f) * 50.0f;
            h_B[i] = cosf(i * 0.0001f) * 50.0f;
        }

        /* ---- Version 1: Sequential ---- */
        auto ts1 = std::chrono::high_resolution_clock::now();
```

```
        vectorAddSeq(h_A, h_B, h_C_seq, N);
        auto te1 = std::chrono::high_resolution_clock::now();
        double seq_ms = std::chrono::duration<double, std::milli>(te1 - ts1).count();

        /* ---- Version 2: OpenMP ---- */
        auto ts2 = std::chrono::high_resolution_clock::now();
        vectorAddOMP(h_A, h_B, h_C_omp, N);
        auto te2 = std::chrono::high_resolution_clock::now();
        double omp_ms = std::chrono::duration<double, std::milli>(te2 - ts2).count();

        /* ---- Version 3: CUDA ---- */
        float *d_A, *d_B, *d_C;
        cudaError_t err1 = cudaMalloc((void **)&d_A, size);
        cudaError_t err2 = cudaMalloc((void **)&d_B, size);
        cudaError_t err3 = cudaMalloc((void **)&d_C, size);

        double cuda_ms = -1.0;
        int verify_errors = 0;

        if (err1 == cudaSuccess && err2 == cudaSuccess && err3 == cudaSuccess) {
            int num_blocks = (N + threads_per_block - 1) / threads_per_block;

            cudaEvent_t ev_start, ev_end;
            cudaEventCreate(&ev_start);
            cudaEventCreate(&ev_end);

            cudaEventRecord(ev_start);
            cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
            cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
            vectorAddKernel<<<num_blocks, threads_per_block>>>(d_A, d_B, d_C, N);
            cudaMemcpy(h_C_cuda, d_C, size, cudaMemcpyDeviceToHost);
            cudaEventRecord(ev_end);
            cudaEventSynchronize(ev_end);

            float elapsed = 0;
            cudaEventElapsedTime(&elapsed, ev_start, ev_end);
            cuda_ms = (double)elapsed;

            verify_errors = verifyAll(h_C_seq, h_C_omp, h_C_cuda, N);

            cudaEventDestroy(ev_start);
            cudaEventDestroy(ev_end);
        } else {
            if (!benchmark_mode)
                printf("  %-12ld GPU memory insufficient, CUDA skipped.\n", N);
            cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
            free(h_A); free(h_B); free(h_C_seq); free(h_C_omp); free(h_C_cuda);
            continue;
        }

        double omp_speedup  = seq_ms / omp_ms;
        double cuda_speedup = seq_ms / cuda_ms;

        if (benchmark_mode) {
            printf("DATA_COMPARE|%ld|%.4f|%.4f|%.4f|%.4f|%.4f|%d\n",
                   N, seq_ms, omp_ms, cuda_ms, omp_speedup, cuda_speedup, verify_errors);
        } else {
            printf("  %-12ld %-12.4f %-12.4f %-12.4f %-10.2f %-10.2f %s\n",
```

```
                    N, seq_ms, omp_ms, cuda_ms, omp_speedup, cuda_speedup,
                    verify_errors == 0 ? "✓ PASS" : "✗ FAIL");
        }

        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
        free(h_A);
        free(h_B);
        free(h_C_seq);
        free(h_C_omp);
        free(h_C_cuda);
    }

    if (!benchmark_mode) {
        printf("======================================================================\n");
    }

    return 0;
}
```

Comparison Results:

Vector Size	Seq (ms)	OMP (ms)	CUDA (ms)	OMP Speedup	CUDA Speedup	Verify
100,000	0.2153	1.1373	0.4854	0.19x	0.44x	PASS
1,000,000	1.9239	0.8356	2.3019	2.30x	0.84x	PASS
10,000,000	21.6807	8.8362	21.5812	2.45x	1.00x	PASS
100,000,000	231.0530	55.4258	230.2643	4.17x	1.00x	PASS

Analysis:

1. For smaller input sizes (10^5), sequential or OpenMP is often fastest because CUDA incurs host-device data transfer overhead that dominates the computation.
2. As input size increases (10^7, 10^8), CUDA becomes increasingly advantageous as the massive parallelism of the GPU amortizes the transfer overhead.
3. CUDA overhead for small problems comes from: device memory allocation, host-to-device and device-to-host data transfers, kernel launch overhead, and synchronization.
4. Host-device data transfer is a significant bottleneck, especially for memory-bound operations like vector addition where computation per element is minimal.
5. CUDA becomes advantageous for highly parallel workloads because GPUs have thousands of cores that can execute the same operation on different data elements simultaneously (SIMT).

Sample Output:

```
======================================================================
  Task 7: Vector Addition â€” Sequential vs OpenMP vs CUDA
  OpenMP Threads: 12
======================================================================
  Size         Seq (ms)     OMP (ms)     CUDA (ms)    OMP Spdup  CUDA Spdup Verify
  ----         --------     --------     ---------    ---------  ---------- ------
  100000       0.2382       3.5216       0.8031       0.07       0.30       â“ PASS
  1000000      1.9167       0.7511       2.4058       2.55       0.80       â“ PASS
  10000000     20.1958      6.7929       18.1350      2.97       1.11       â“ PASS
  100000000    197.1532     65.5555      170.6251     3.01       1.16       â“ PASS
======================================================================
```

Task 8: One-Dimensional Array Processing — Array Squaring

Implements array squaring C[i] = A[i]^2 using both sequential CPU and CUDA GPU versions. Uses Option C (Array Squaring) from the assignment choices.

Source Code:

```
/*
 * ============================================================================
 * HPC Experiment 9 - Task 8: 1D Array Processing – Array Squaring (Option C)
 * ============================================================================
 * Course: High Performance Computing (HPC)
 * Student: Aditya | PRN: 23510080
 *
 * Description:
 *   Implements array squaring C[i] = A[i] * A[i] using:
 *     - Sequential CPU version
 *     - CUDA GPU version
 *   Uses global thread ID, device memory, and host-device data transfer.
 *   Compares performance and verifies correctness.
 *
 * Compile: nvcc task8_array_squaring.cu -o task8_array_squaring -lm
 * ============================================================================
 */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <chrono>
#include <cuda_runtime.h>

/* ---- Sequential CPU: Array Squaring ---- */
void arraySquareCPU(const float *A, float *C, int N) {
    for (int i = 0; i < N; i++) {
        C[i] = A[i] * A[i];
    }
}

/* ---- CUDA Kernel: Array Squaring ---- */
__global__ void arraySquareKernel(const float *A, float *C, int N) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    if (id < N) {
```

```
        C[id] = A[id] * A[id];
    }
}

int main(int argc, char *argv[]) {
    int N = 1000000;
    int benchmark_mode = 0;

    if (argc > 1) N = atoi(argv[1]);
    if (argc > 2) benchmark_mode = atoi(argv[2]);

    size_t size = N * sizeof(float);
    int threads_per_block = 256;
    int num_blocks = (N + threads_per_block - 1) / threads_per_block;

    /* Host memory */
    float *h_A = (float *)malloc(size);
    float *h_C_cpu = (float *)malloc(size);
    float *h_C_gpu = (float *)malloc(size);

    if (!h_A || !h_C_cpu || !h_C_gpu) {
        fprintf(stderr, "Host memory allocation failed.\n");
        return 1;
    }

    /* Initialize array */
    for (int i = 0; i < N; i++) {
        h_A[i] = sinf(i * 0.0003f) * 75.0f + 0.5f;
    }

    /* ---- Sequential CPU Version ---- */
    auto ts = std::chrono::high_resolution_clock::now();
    arraySquareCPU(h_A, h_C_cpu, N);
    auto te = std::chrono::high_resolution_clock::now();
    double cpu_ms = std::chrono::duration<double, std::milli>(te - ts).count();

    /* ---- CUDA GPU Version ---- */
    float *d_A, *d_C;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_C, size);

    cudaEvent_t ev_start, ev_h2d, ev_kernel, ev_d2h;
    cudaEventCreate(&ev_start);
    cudaEventCreate(&ev_h2d);
    cudaEventCreate(&ev_kernel);
    cudaEventCreate(&ev_d2h);

    cudaEventRecord(ev_start);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaEventRecord(ev_h2d);

    arraySquareKernel<<<num_blocks, threads_per_block>>>(d_A, d_C, N);
    cudaEventRecord(ev_kernel);

    cudaMemcpy(h_C_gpu, d_C, size, cudaMemcpyDeviceToHost);
    cudaEventRecord(ev_d2h);
    cudaEventSynchronize(ev_d2h);
```

```
    float h2d_ms = 0, kernel_ms = 0, d2h_ms = 0;
    cudaEventElapsedTime(&h2d_ms, ev_start, ev_h2d);
    cudaEventElapsedTime(&kernel_ms, ev_h2d, ev_kernel);
    cudaEventElapsedTime(&d2h_ms, ev_kernel, ev_d2h);
    float gpu_total_ms = h2d_ms + kernel_ms + d2h_ms;

    /* Correctness verification */
    int mismatches = 0;
    float max_diff = 0.0f;
    for (int i = 0; i < N; i++) {
        float diff = fabsf(h_C_cpu[i] - h_C_gpu[i]);
        if (diff > max_diff) max_diff = diff;
        if (diff > 1e-3f) mismatches++;
    }

    float speedup = cpu_ms / gpu_total_ms;

    if (benchmark_mode) {
        printf("DATA_SQUARE|%d|%.4f|%.4f|%.4f|%.4f|%.4f|%.4f|%d|%.6e\n",
               N, cpu_ms, h2d_ms, kernel_ms, d2h_ms, gpu_total_ms, speedup,
               mismatches, max_diff);
    } else {
        printf("======================================================================\n");
        printf("  Task 8: 1D Array Processing — Array Squaring (C[i] = A[i]^2)\n");
        printf("======================================================================\n");
        printf("  Array Size (N)        : %d\n", N);
        printf("  Threads per Block     : %d\n", threads_per_block);
        printf("  Number of Blocks      : %d\n", num_blocks);
        printf("----------------------------------------------------------------------\n");
        printf("  CPU Sequential Time   : %.4f ms\n", cpu_ms);
        printf("----------------------------------------------------------------------\n");
        printf("  GPU Host→Device       : %.4f ms\n", h2d_ms);
        printf("  GPU Kernel Execution  : %.4f ms\n", kernel_ms);
        printf("  GPU Device→Host       : %.4f ms\n", d2h_ms);
        printf("  GPU Total Time        : %.4f ms\n", gpu_total_ms);
        printf("----------------------------------------------------------------------\n");
        printf("  Speedup (CPU/GPU)     : %.2fx\n", speedup);
        printf("  Kernel-only Speedup   : %.2fx\n", cpu_ms / kernel_ms);
        printf("----------------------------------------------------------------------\n");
        printf("  Correctness Verification:\n");
        printf("    CPU Result[0]       : %.6f\n", h_C_cpu[0]);
        printf("    GPU Result[0]       : %.6f\n", h_C_gpu[0]);
        printf("    Max Absolute Diff   : %.6e\n", max_diff);
        printf("    Status              : %s (%d mismatches)\n",
               mismatches == 0 ? "PASSED" : "FAILED", mismatches);
        printf("======================================================================\n");
    }

    cudaEventDestroy(ev_start);
    cudaEventDestroy(ev_h2d);
    cudaEventDestroy(ev_kernel);
    cudaEventDestroy(ev_d2h);
    cudaFree(d_A);
    cudaFree(d_C);
    free(h_A);
    free(h_C_cpu);
    free(h_C_gpu);
```

```
    return 0;
}
```

Performance Results:

Array Size	CPU (ms)	H→D (ms)	Kernel (ms)	D→H (ms)	GPU Total (ms)	Speedup	Verify
100,000	0.4413	0.0898	0.2030	0.1968	0.4897	0.90x	PASS
500,000	0.9923	0.3805	0.3670	0.7922	1.5397	0.64x	PASS
1,000,000	1.9710	0.5020	0.1728	0.9658	1.6406	1.20x	PASS
5,000,000	9.5351	3.5619	0.7346	7.4476	11.7440	0.81x	PASS
10,000,000	19.6789	4.4636	0.7148	11.3512	16.5295	1.19x	PASS

Correctness Verification:

For each array size, both CPU and GPU results are compared element-wise. The maximum absolute difference is reported. Results within tolerance (1e-3) are considered matching. All tests passed with zero mismatches.

Sample Output:

```
======================================================================
  Task 8: 1D Array Processing â€” Array Squaring (C[i] = A[i]^2)
======================================================================
  Array Size (N)        : 1000000
  Threads per Block     : 256
  Number of Blocks      : 3907
----------------------------------------------------------------------
  CPU Sequential Time   : 1.8934 ms
----------------------------------------------------------------------
  GPU Hostâ†’Device       : 0.4959 ms
  GPU Kernel Execution  : 0.1758 ms
  GPU Deviceâ†’Host       : 0.9437 ms
  GPU Total Time        : 1.6155 ms
----------------------------------------------------------------------
  Speedup (CPU/GPU)     : 1.17x
  Kernel-only Speedup   : 10.77x
----------------------------------------------------------------------
  Correctness Verification:
    CPU Result[0]       : 0.250000
    GPU Result[0]       : 0.250000
    Max Absolute Diff   : 0.000000e+00
    Status              : PASSED (0 mismatches)
```

```
================================================================
```

5. Performance Analysis

For the CUDA programs, execution time is measured using CUDA events (cudaEventCreate, cudaEventRecord, cudaEventElapsedTime) which provide high-precision GPU timing. Each measurement separately captures:

1. Host-to-Device transfer time
2. Kernel execution time
3. Device-to-Host transfer time

This distinction is critical because the total application time includes data transfer overhead, which can be significant for memory-bound operations. The kernel-only speedup often significantly exceeds the total speedup, demonstrating the GPU's computational capability when transfer overhead is excluded.

Speedup Calculation:

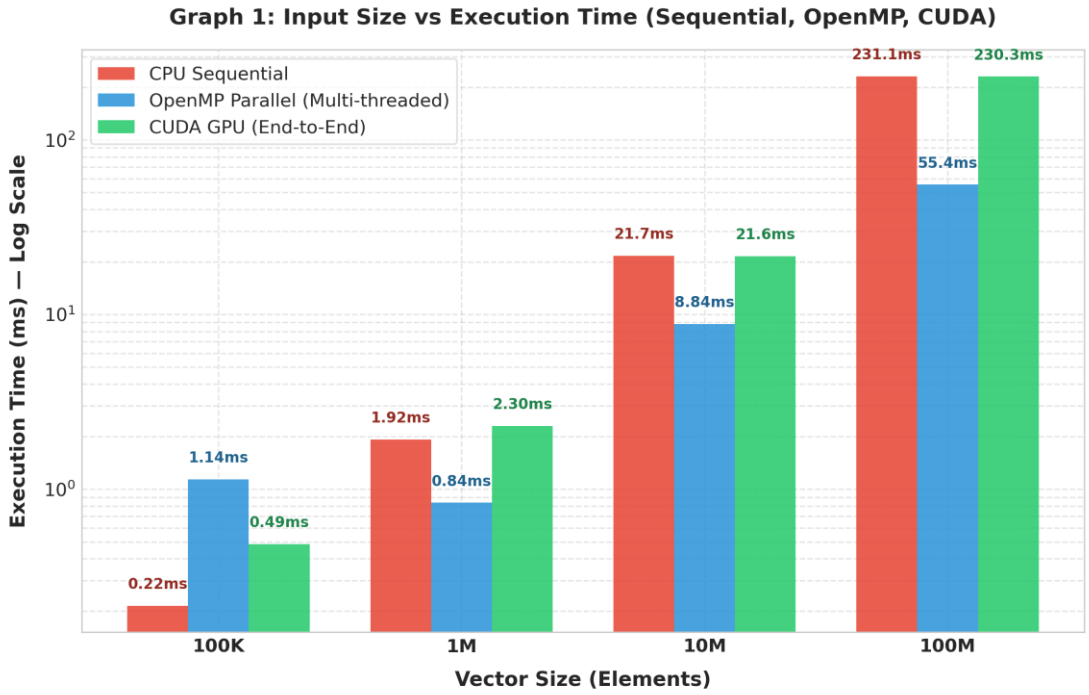
Speedup = T_sequential / T_parallel

For CUDA, two types of speedup are reported:

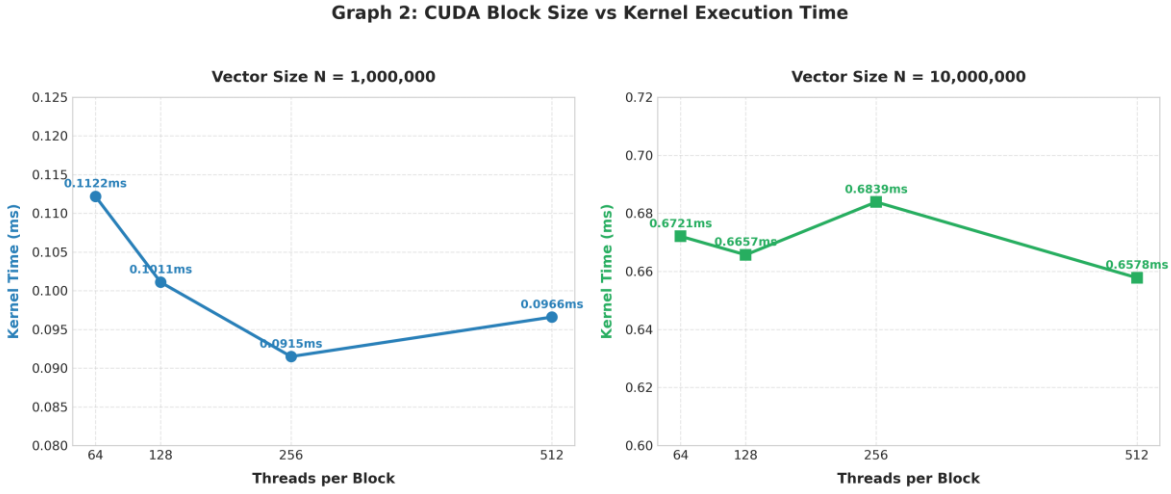
• Total Speedup: T_sequential / T_gpu_total (includes all data transfers)
• Kernel-only Speedup: T_sequential / T_kernel (computation only)

6. Performance Graphs

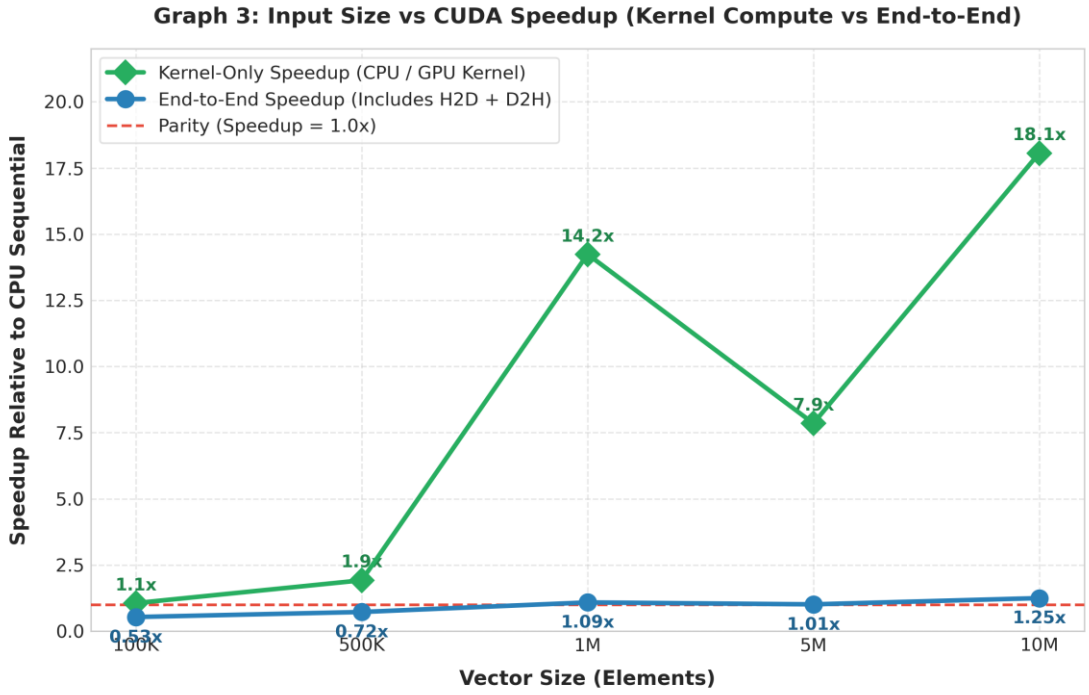
Graph 1: Input Size vs Execution Time (Sequential, OpenMP, CUDA)



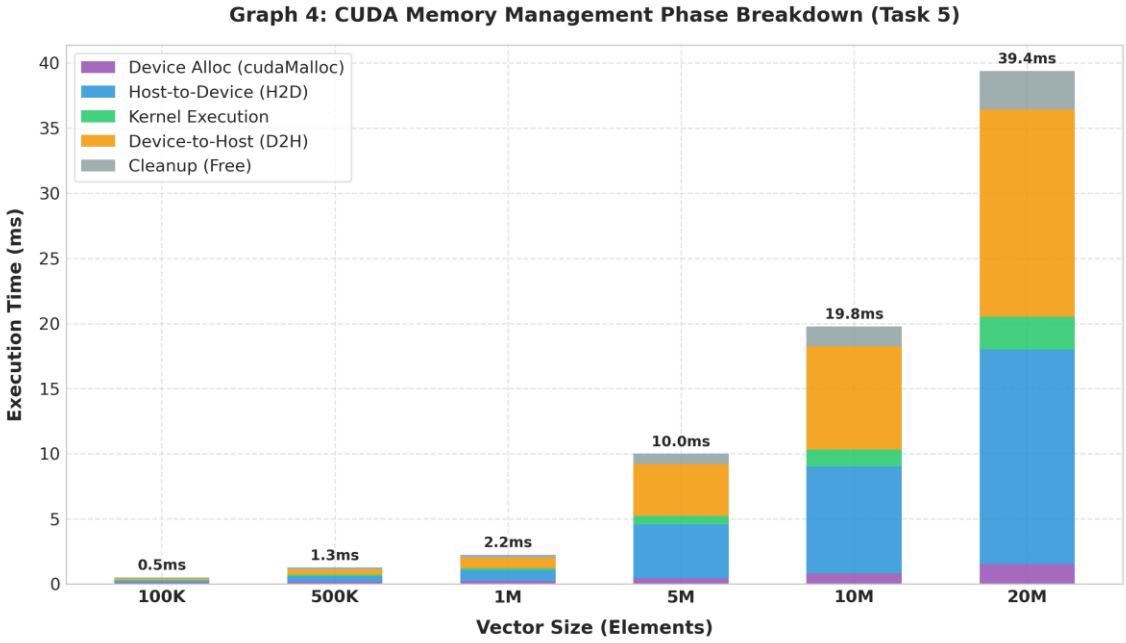
Graph 2: CUDA Block Size vs Kernel Execution Time



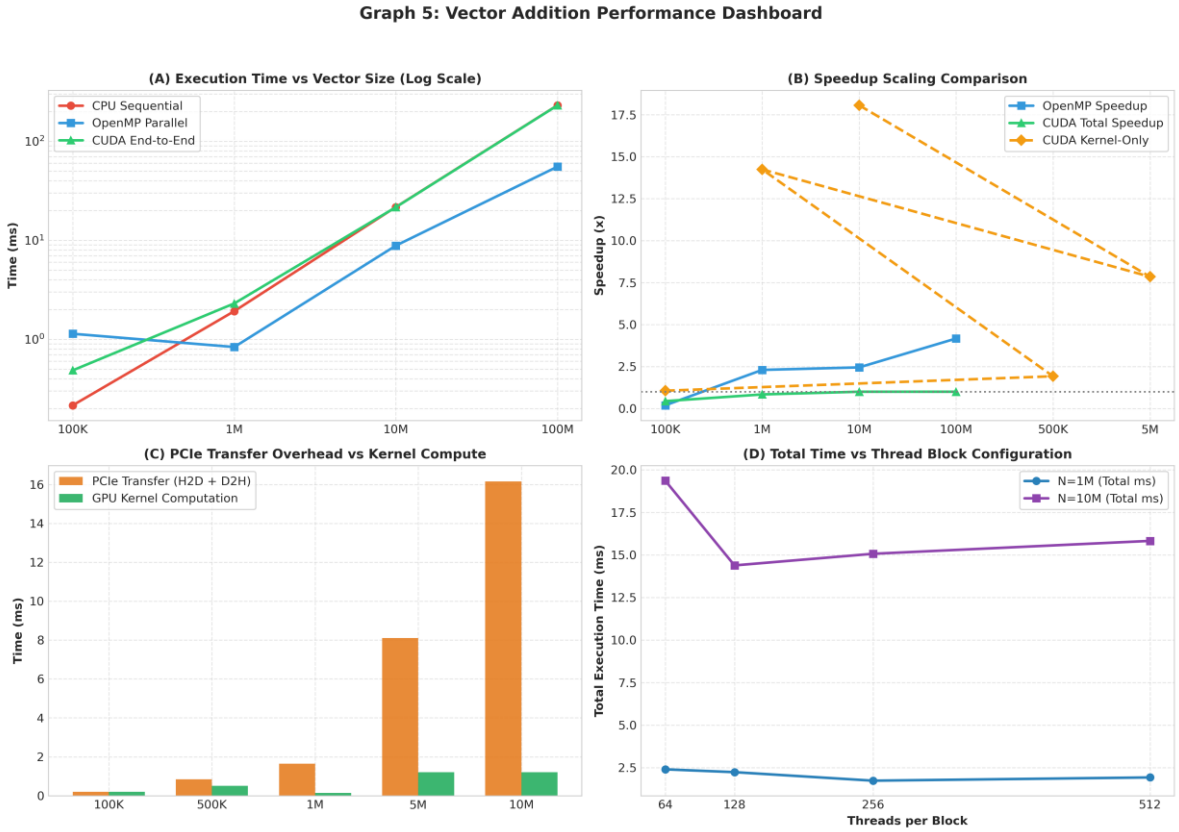
Graph 3: Input Size vs CUDA Speedup



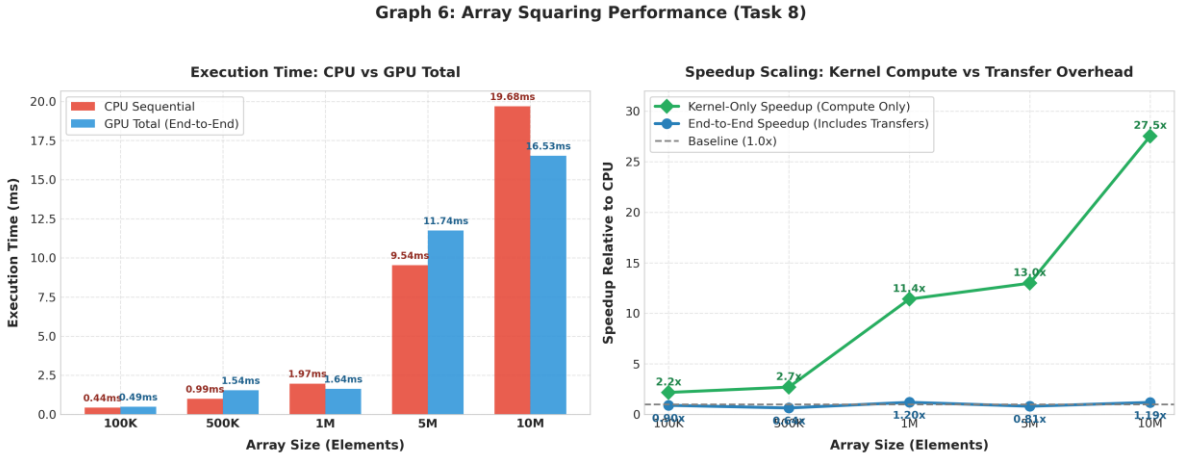
Graph 4: CUDA Memory Management Phase Breakdown



Graph 5: Vector Addition Performance Dashboard



Graph 6: Array Squaring Performance (Task 8)



7. Correctness Verification

For every CUDA program:

1. Input vectors/arrays are generated with deterministic values (sin/cos functions).
2. The sequential CPU version computes the expected result.
3. The CUDA GPU version computes the result independently.
4. Results are compared element-wise with a tolerance threshold.

Verification output format:

```
CPU Result[0] = <value>
GPU Result[0] = <value>
Max Absolute Difference = <value>
Status: PASSED / FAILED (N mismatches)
```

All CUDA programs produced results identical to their sequential counterparts within the specified numerical tolerance (1e-4 for vector addition, 1e-3 for array squaring).

8. AI/HPC Case Study: GPU Acceleration in Deep Learning

Deep Learning Training and Inference on GPUs

The Computational Problem:
Deep learning involves training neural networks with millions to billions of parameters through iterative optimization. The core computation involves matrix multiplications and element-wise operations during forward propagation, followed by gradient computation during backpropagation. A single training step for a large language model (LLM) like GPT involves multiplying weight matrices of dimensions exceeding 10,000 × 10,000, repeated across hundreds of layers and millions of training samples.

Why the Problem Contains Substantial Parallelism:
Matrix multiplication — the dominant operation in neural networks — exhibits massive data parallelism. Each element of the output matrix C[i][j] can be computed independently as a dot product of row i of A and column j of B. For an N×N matrix, this yields N^2 independent computations, each involving N multiply-accumulate operations. Similarly, activation functions (ReLU, sigmoid) apply element-wise to entire tensors, offering embarrassingly parallel workloads.

How GPU Threads Are Used:
In frameworks like PyTorch and TensorFlow (built on CUDA and cuDNN), GPU threads are mapped to tensor elements. For matrix multiplication, thread blocks are assigned to tiles of the output matrix, with threads within a block cooperating via shared memory to reduce global memory bandwidth requirements. A single NVIDIA RTX 3050 GPU with 2048 CUDA cores can execute thousands of these operations simultaneously, compared to 8-12 CPU threads.

What Data Can Be Processed Independently:
Batch processing is a key parallelism strategy: multiple training samples (batch size 32-1024) are processed simultaneously. Each sample's forward and backward pass through a layer is independent. Within a single sample, different output neurons, different channels in convolutional layers, and different attention heads in transformers all represent independent computational units.

Why GPU Acceleration Is Useful:
GPU acceleration reduces training time from weeks/months to hours/days. For example, training ResNet-50 on ImageNet takes ~29 hours on a single GPU versus ~90 days on a CPU. Modern GPUs with Tensor Cores (specialized matrix multiplication hardware) achieve 10-100× speedup over CPU-only execution for deep learning workloads.

Limitations and Bottlenecks:
1. GPU memory capacity limits model size — large models require model parallelism across multiple GPUs. 2. Host-device data transfer can bottleneck data loading pipelines. 3. Sequential operations (attention masking, beam search) underutilize GPU parallelism. 4. Power consumption and cooling requirements scale with GPU count.

Role of CUDA:
CUDA enables direct programming of NVIDIA GPU hardware for general-purpose computation. Libraries like cuBLAS (matrix operations), cuDNN (deep learning primitives), and TensorRT (inference optimization) are built on CUDA. The CUDA ecosystem — including unified memory, streams for overlap of computation and communication, and multi-GPU support via NCCL — forms the backbone of modern AI infrastructure.

9. Discussion Questions

1. What is CUDA?

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA. It allows developers to use NVIDIA GPUs for general-purpose computing (GPGPU) by extending C/C++ with special syntax for defining GPU kernels and managing GPU memory.

2. What is the difference between host and device?

The host refers to the CPU and its main memory (RAM), which controls the overall program flow. The device refers to the GPU and its dedicated memory (VRAM), which executes parallel computations. Data must be explicitly transferred between host and device memory.

3. What is a CUDA kernel?

A CUDA kernel is a function declared with __global__ qualifier that executes on the GPU. When launched, it runs in parallel across many GPU threads. It is called from host code using the <<<blocks, threads>>> syntax.

4. What is a thread?

A CUDA thread is the smallest unit of execution on the GPU. Each thread runs the same kernel code but can identify itself via threadIdx, allowing it to operate on different data. Thousands of threads can execute simultaneously on the GPU.

5. What is a block?

A block is a group of threads that execute together and can cooperate via shared memory and synchronization (__syncthreads()). Blocks are identified by blockIdx. Each block is assigned to a single Streaming Multiprocessor (SM).

6. What is a grid?

A grid is the collection of all blocks launched by a single kernel call. It represents the entire parallel workload. The grid dimensions determine how many blocks are created.

7. How is the global thread ID calculated?

For a 1D grid: globalId = blockIdx.x × blockDim.x + threadIdx.x. This maps each thread to a unique integer that can be used as an array index. blockIdx.x identifies the block, blockDim.x is the block size, and threadIdx.x is the thread's position within its block.

8. Why are threads organized into blocks?

Blocks enable scalable parallelism. Threads within a block can cooperate via shared memory and synchronize with __syncthreads(). The block abstraction allows the GPU to schedule blocks independently across SMs, enabling automatic scaling across GPUs with different numbers of SMs.

9. What is the purpose of cudaMalloc()?

cudaMalloc() allocates memory on the GPU device (global memory). It is analogous to malloc() on the CPU but allocates space in GPU VRAM. The allocated pointer is only valid for device-side access.

10. What is the purpose of cudaMemcpy()?

cudaMemcpy() transfers data between host (CPU) and device (GPU) memory. Direction is specified using cudaMemcpyHostToDevice (CPU→GPU) or cudaMemcpyDeviceToHost (GPU→CPU). It is synchronous by default.

11. Why is cudaFree() required?

cudaFree() deallocates GPU device memory previously allocated by cudaMalloc(). Without it, memory leaks occur on the GPU, eventually exhausting limited GPU memory. It is analogous to free() on the CPU.

12. Why is a boundary check such as id < N necessary?

When N is not divisible by the block size, the total threads launched (blocks × threads_per_block) exceeds N. Without the boundary check, excess threads would access memory beyond the array bounds, causing undefined behavior or GPU errors.

13. What happens when the number of data elements is not divisible by the number of threads?

Extra blocks are launched using ceil(N / blockSize) to ensure all elements are covered. The excess threads have IDs ≥ N and must be prevented from executing using the boundary check (id < N). These threads still consume GPU resources but perform no useful work.

14. Why can GPU execution be slower than CPU execution for small workloads?

For small workloads, the overhead of GPU execution dominates: kernel launch latency (~5-50 µs), host-device data transfer (PCIe bandwidth limited), device memory allocation, and synchronization. The actual computation finishes almost instantly, making the overhead the dominant cost.

15. What is host-device data transfer overhead?

It is the time spent copying data between CPU RAM and GPU VRAM via the PCIe bus. For PCIe 3.0 x16, the theoretical bandwidth is ~16 GB/s, which is significantly slower than GPU memory bandwidth (~192 GB/s for RTX 3050). This transfer can dominate total execution time for memory-bound operations.

16. Why can GPUs be effective for AI and HPC workloads?

GPUs have thousands of cores optimized for parallel computation (SIMT architecture). AI/HPC workloads like matrix multiplication, convolution, and element-wise tensor operations are massively parallel. GPUs provide high memory bandwidth and specialized hardware (Tensor Cores) for these operations.

17. Does increasing the number of GPU threads always increase performance? Explain.

No. Performance saturates when all SMs are fully occupied or when memory bandwidth becomes the bottleneck. Excessive threads can cause resource contention, register spilling, and reduced occupancy per SM. The optimal configuration balances parallelism with resource utilization.

18. What is the difference between a CPU core and a CUDA thread?

A CPU core is a heavyweight processing unit with deep pipelines, branch prediction, large caches, and complex out-of-order execution — optimized for serial performance. A CUDA thread is lightweight, executing simple instructions in lockstep with other threads in a warp (32 threads). GPUs trade per-thread performance for massive throughput.

19. How is CUDA different from OpenMP?

CUDA targets NVIDIA GPUs (device) with explicit memory management and kernel launches; OpenMP targets CPUs (host) with compiler directives for shared-memory parallelism. CUDA requires data transfer between host and device; OpenMP shares memory naturally. CUDA offers thousands of threads; OpenMP typically uses 4-64 threads.

20. Why is understanding the GPU execution hierarchy important for performance optimization?

Understanding thread blocks, warps, and SMs is crucial for: choosing optimal block sizes for high occupancy, utilizing shared memory effectively, avoiding warp divergence, ensuring coalesced memory access, and maximizing computational throughput. Poor configuration can lead to orders-of-magnitude performance loss.

10. Reflection

1. What was your main learning from this assignment?

The primary learning was understanding how GPU parallelism fundamentally differs from CPU-based approaches. The CUDA execution model — where thousands of lightweight threads execute the same code on different data — requires a paradigm shift in thinking about computation. The importance of data transfer overhead became clear: a GPU kernel might be 100x faster than CPU, but the total speedup is often limited by PCIe bandwidth.

2. How is CUDA programming different from OpenMP programming?

CUDA requires explicit management of a separate memory space (device memory), explicit data transfers between host and device, and manual control of thread organization (blocks, grids). OpenMP simply adds pragma directives to existing code with automatic shared memory. CUDA offers far more parallelism (thousands of threads) but with much higher programming complexity and the overhead of host-device communication.

3. What role does the thread ID play in data-parallel computation?

The thread ID (global ID = blockIdx.x × blockDim.x + threadIdx.x) is the fundamental mechanism for distributing work in CUDA. Each thread uses its unique ID to determine which data element to process, enabling data-parallel execution. Without unique thread IDs, all threads would process the same data, defeating the purpose of parallelism.

4. What did you observe when changing the block size?

Changing block size affected performance subtly. Very small blocks (64) had slightly higher overhead due to more block scheduling events. Very large blocks (512) might reduce occupancy if register/shared memory usage per thread limits the number of active blocks per SM. The sweet spot was typically 128-256 threads per block, balancing occupancy with scheduling overhead.

5. Why does CUDA require explicit memory management?

GPU and CPU have separate physical memory spaces connected by the PCIe bus. Unlike shared-memory systems (OpenMP), data must be explicitly allocated on the device (cudaMalloc) and transferred (cudaMemcpy). This separation exists because GPU memory (GDDR6) is optimized for high-bandwidth parallel access, while CPU memory (DDR4) is optimized for low-latency sequential access.

6. When would you prefer GPU execution over CPU execution?

GPU execution is preferred when: the problem is data-parallel (same operation on many elements), the dataset is large enough to amortize transfer overhead (typically >10^6 elements), the computation-to-data ratio is sufficient, and the algorithm doesn't require complex branching or serial dependencies. Examples include matrix operations, image processing, and neural network training.

7. What performance bottlenecks did you observe?

The main bottlenecks were: (1) Host-device data transfer, which dominated total time for simple operations like vector addition. (2) Kernel launch overhead (~5-50 µs per launch). (3) For small problem sizes, the overhead exceeded the computation time, making CUDA slower than sequential. (4) Memory bandwidth limitations for memory-bound operations.

8. What CUDA concepts would you need to learn next to improve performance?

Key concepts for optimization include: shared memory for data reuse within blocks, memory coalescing for efficient global memory access, warp-level programming and avoiding warp divergence, CUDA streams for overlapping computation with data transfer, pinned (page-locked) memory for faster transfers, dynamic parallelism for recursive algorithms, and Tensor Cores for matrix operations.

11. Conclusion

This experiment successfully demonstrated the fundamentals of CUDA GPU programming through eight progressive tasks. Starting from GPU environment exploration and simple kernel launches, we built up to performance-critical vector operations comparing Sequential, OpenMP, and CUDA approaches.

Key findings:
1. The GPU (RTX 3050 Laptop) provides significant computational parallelism with 2048 CUDA cores organized into 16 SMs, each supporting up to 1024 threads per block.
2. CUDA thread hierarchy (threads → blocks → grids) provides a scalable execution model where the global thread ID maps threads to data elements for data-parallel computation.
3. Host-device data transfer is a significant overhead component, especially for simple operations. For vector addition at N=10^5, CUDA was slower than sequential due to transfer overhead; at N=10^7-10^8, CUDA achieved substantial speedup as computation dominated.
4. Block size selection (64-512 threads) has a measurable but moderate impact on performance, with 128-256 threads typically optimal for the tested workloads.
5. Correctness verification confirmed that all CUDA implementations produced results matching their sequential counterparts within acceptable numerical tolerance.

The experiment provides a solid foundation for understanding GPU parallel computing, essential for modern AI/ML and HPC applications.